

Para el servicio de python

```
sudo apt update
```

```
sudo apt install -y tesseract-ocr tesseract-ocr-spa ghostscript qpdf pngquant unpaper
```

```
install -y python3 python3-venv python3-pip
```

```
python3 -m venv .venv
```

```
source .venv/bin/activate
```

```
pip install --upgrade pip setuptools wheel
```

```
pip install fastapi uvicorn python-multipart pypdf ocrmypdf
```

```
sudo vim /etc/systemd/system/ocr-python.service
```

[Unit]

Description=Servicio OCR Python FastAPI

After=network.target

[Service]

Type=simple

User=benja

WorkingDirectory=/home/benja/ocr_service

ExecStart=/home/benja/ocr_service/.venv/bin/uvicorn app:app --host 127.0.0.1 --port 8001

Restart=always

RestartSec=3

[Install]

WantedBy=multi-user.target

```
sudo systemctl daemon-reload
```

```
sudo systemctl enable ocr-python.service
```

```
sudo systemctl start ocr-python.service
```

```
#app.py
```

```
#!/usr/bin/env python3
```

```
import sys
```

```
import uuid
```

```
import shutil
```

```
import asyncio
```

```
import tempfile
```

```
import subprocess
```

```
from pathlib import Path
```

```
from fastapi import FastAPI, UploadFile, File, HTTPException, BackgroundTasks
```

```
from fastapi.responses import FileResponse
```

```
from pypdf import PdfReader
```

```
app = FastAPI(title="OCR PDF Service")
```

```
def pdf_tiene_texto(ruta_pdf: Path, paginas_a_revisar: int = 5, min_chars: int = 20) -> bool:
```

```
    reader = PdfReader(str(ruta_pdf))
```

```
    total = len(reader.pages)
```

```
    limite = min(total, paginas_a_revisar)
```

```
    for i in range(limite):
```

```
        try:
```

```
            texto = reader.pages[i].extract_text()
```

```
            if texto and len(texto.strip()) >= min_chars:
```

```
                return True
```

```
        except Exception:
```

```
            continue
```

```
    return False
```

```
def ejecutar_ocr(entrada: Path, salida: Path, idioma: str = "spa") -> tuple[int, str]:
```

```
    cmd = [
```

```
        sys.executable,
```

```
        "-m",
```

```
        "ocrmypdf",
```

```
        "-l",
```

```
        idioma,
```

```
        "-m",
```

```
        "skip",
```

```
        "--deskew",
```

```
        "--clean",
```

```
        str(entrada),
```

```
        str(salida),
```

```
    ]
```

```
    proc = subprocess.run(
```

```
        cmd,
```

```
        stdout=subprocess.PIPE,
```

```
        stderr=subprocess.STDOUT,
```

```
        text=True
```

```
    )
```

```
    return proc.returncode, proc.stdout
```

```
def borrar_directorio(ruta: str):
```

```
    shutil.rmtree(ruta, ignore_errors=True)
```

```
@app.get("/health")
```

```
async def health():
```

```
return {"status": "ok"}
```

```
@app.post("/ocr")
async def ocr_pdf(
    background_tasks: BackgroundTasks,
    file: UploadFile = File(...)
):
    if not file.filename:
        raise HTTPException(status_code=400, detail="No se recibió archivo")

    if not file.filename.lower().endswith(".pdf"):
        raise HTTPException(status_code=400, detail="El archivo debe ser PDF")

    temp_dir = tempfile.mkdtemp(prefix="ocrsvc_")
    entrada = Path(temp_dir) / f"entrada_{uuid.uuid4().hex}.pdf"
    salida = Path(temp_dir) / f"salida_{uuid.uuid4().hex}.pdf"

    try:
        with open(entrada, "wb") as f:
            contenido = await file.read()
            f.write(contenido)

    try:
        ya_tiene_texto = pdf_tiene_texto(entrada)
    except Exception as e:
        background_tasks.add_task(borrar_directorio, temp_dir)
        raise HTTPException(status_code=500, detail=f"No se pudo analizar el PDF: {e}")

    if ya_tiene_texto:
        background_tasks.add_task(borrar_directorio, temp_dir)
        return FileResponse(
            path=str(entrada),
            media_type="application/pdf",
            filename=file.filename,
            headers={
                "X-OCR-Status": "already_has_text"
            }
        )

    codigo, log = await asyncio.to_thread(ejecutar_ocr, entrada, salida, "spa")

    if codigo != 0:
        background_tasks.add_task(borrar_directorio, temp_dir)
        raise HTTPException(status_code=500, detail=f"Error al ejecutar
OCRmyPDF:\n{log}")

    if not salida.exists():

```

```
        background_tasks.add_task(borrar_directorio, temp_dir)
        raise HTTPException(status_code=500, detail="OCRmyPDF terminó pero no generó
salida")
```

```
        background_tasks.add_task(borrar_directorio, temp_dir)
```

```
    return FileResponse(
        path=str(salida),
        media_type="application/pdf",
        filename=f"{Path(file.filename).stem}_ocr.pdf",
        headers={
            "X-OCR-Status": "ocr_generated"
        }
    )
```

```
except HTTPException:
```

```
    raise
```

```
except Exception as e:
```

```
    background_tasks.add_task(borrar_directorio, temp_dir)
```

```
    raise HTTPException(status_code=500, detail=str(e))
```